

INTRODUCTION TO RISC-V

RISC-V入门教程

ISA | 汇编指令 | 系统编程 | 组成原理 | 嵌入式应用

异常、中断与特权模式

主讲 邢建国



中国开放指令生态 (RISC-V) 联盟 | 浙江中心
China RISC-V Alliance | Zhejiang Center



浙江图灵算力研究院
ZHEJIANG TURING INSTITUTE





异常、中断和特权模式



- QEMU
- 在QEMU中运行程序
- 特权模式
- 异常
- 定时器中断

■ 中断和异常

许多计算机系统被组织为用户软件和系统软件。

系统软件（操作系统，例如系统内核和设备驱动程序）是负责保护和管理整个系统的软件，包括与外围设备交互以执行输入和输出操作，以及加载和调度用户应用程序以执行。

用户软件是通过操作系统来访问外设和其他资源。

RISCV中提供了大量的保护系统软件 and 用户软件的机制，包括特权模式、异常和中断等。

本节讨论保护系统免受错误的异常和中断硬件机制以及如何编程。

■ QEMU

- QEMU (Quick Emulator) 是一个开源的硬件虚拟化工具
 - 既可以作为系统模拟器（全系统模拟），也可以作为用户态程序模拟器（单程序模拟）。
 - 支持多种处理器架构（如x86、ARM、RISC-V、MIPS等）
 - qemu
 - 支持常见外设（磁盘、网络、GPU、USB等）
 - 允许用户在不同的主机平台上运行未修改的操作系统或程序。
- Renode
 - Renode 是由 Antmicro 开发的一个开源仿真和虚拟开发框架，用于加速物联网 (IoT) 和嵌入式系统的开发。它通过模拟物理硬件系统，包括 CPU、外围设备、传感器、环境以及节点之间的有线或无线通信，帮助开发者在 PC 上运行、调试和测试未经修改的嵌入式软件

■ QEMU对RISC-V的支持

- 支持的RISC-V处理器特性
 - 核心指令集
 - RV32I/RV64I（基础整数指令）、RV32E（嵌入式精简版）、扩展指令集（M乘除、A原子操作、F/D浮点、C压缩指令等）。
 - 特权级架构
 - 支持机器模式（M-mode）、监管模式（S-mode）、用户模式（U-mode），可模拟操作系统运行环境。
 - 扩展支持
 - 逐步支持V扩展（向量指令）、H扩展（虚拟化）、Zicsr（控制状态寄存器）等新特性。
- 多核模拟：支持多核RISC-V CPU的模拟（如SMP系统）。

■ QEMU对RISC-V的支持

- 支持的RISC-V开发板与设备
 - 预定义开发板模型
 - 如virt（通用虚拟开发板）、sifive_u（SiFive U系列开发板）、microchip-polarfire等。
 - 外设模拟
 - 包括UART、CLINT（核心本地中断器）、PLIC（平台级中断控制器）、VirtIO设备（磁盘、网络、GPU）、PCIe总线等。
 - 启动方式
 - 支持直接加载RISC-V内核镜像（如Linux的Image文件）、设备树（DTB）或通过Bootloader（如U-Boot）启动。

■ QEMU对RISC-V的支持

- 操作系统兼容性
 - Linux：完全支持运行RISC-V Linux发行版（如Fedora、Debian、OpenSUSE）。
 - 实时操作系统（RTOS）：如FreeRTOS、Zephyr等。
 - 裸机程序：可直接运行RISC-V汇编或C语言编写的裸机代码（如嵌入式固件）。
- 调试与开发工具链
 - GDB集成：通过-s -S参数启动QEMU内置的GDB调试服务器，支持单步调试内核或应用程序。
 - RISC-V工具链兼容：与主流的RISC-V编译器（如riscv64-unknown-elf-gcc）和调试工具无缝协作。

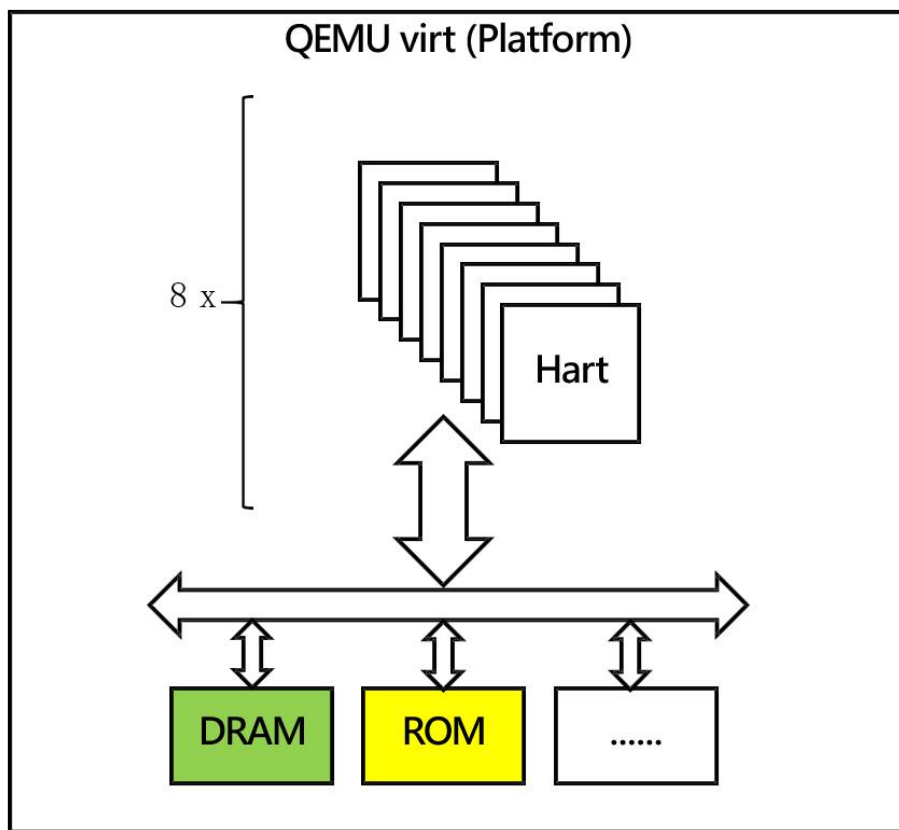
■ qemu-system-riscv32安装

- ubuntu 24下安装qemu-system-riscv32
 - sudo apt update
 - sudo apt install qemu-system-misc
 - 这个命令会安装 QEMU 的 RISC-V 模拟器，包括 qemu-system-riscv32 和 qemu-system-riscv64
- 检查 QEMU 是否安装成功
 - qemu-system-riscv32 -version
 - 如果输出 QEMU 的版本信息，则表示安装成功

```
xing@xinghz:~/chap4/4-3-old$ qemu-system-riscv32 -version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.4)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project developers
```


■ VIRT

- QEMU RISC-V Virt 机器是一个通用的虚拟平台，它支持多种设备和虚拟化扩展，提供了灵活的配置选项，适用于 RISC-V 软件开发和测试。



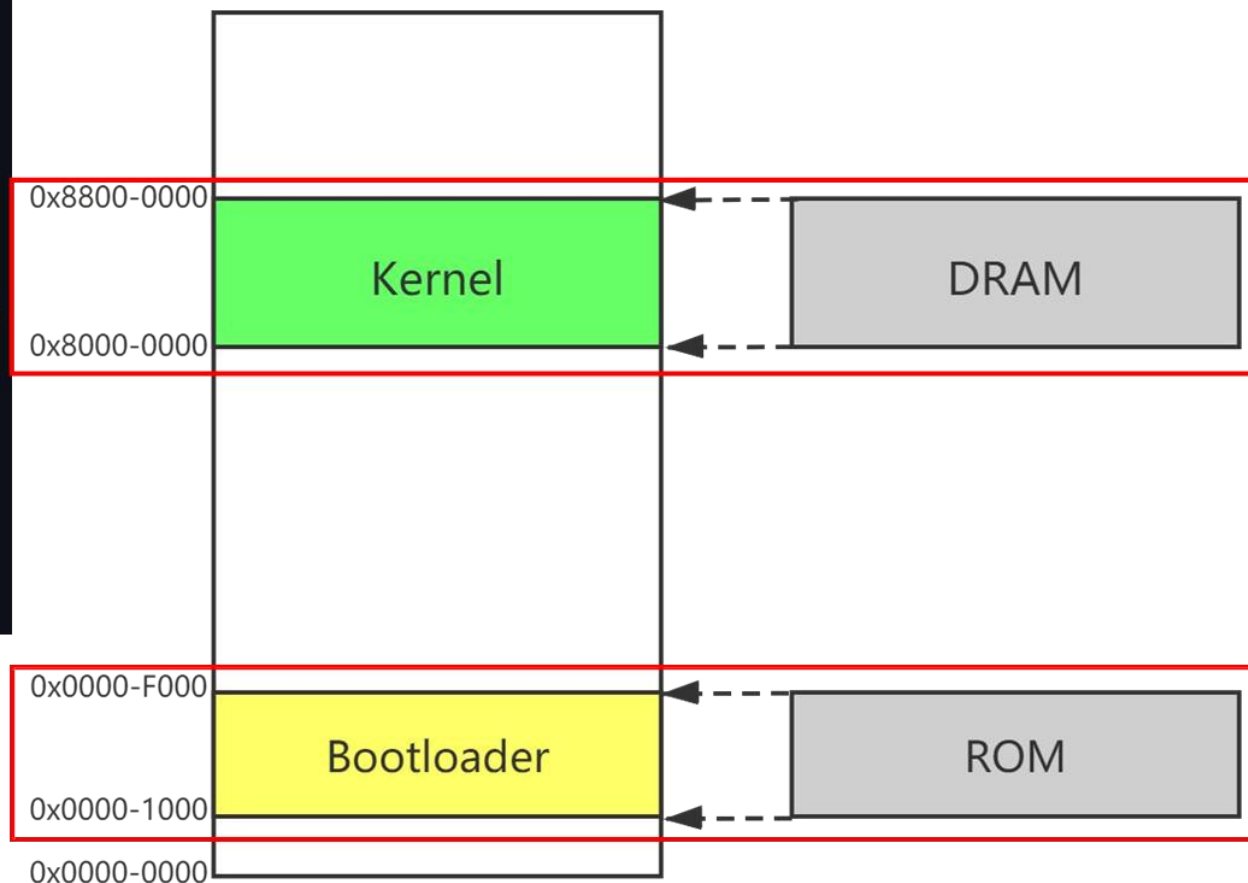
■ VIRT

- QEMU RISC-V Virt 支持多种设备，包括：
 - CPU 核心：支持多达 512 个通用的 RV32GC/RV64GC 核心，可选扩展。
 - CLINT (Core Local Interruptor)：核心本地中断控制器。
 - PLIC (Platform-Level Interrupt Controller)：平台级中断控制器。
 - CFI 并行 NOR 闪存：用于存储固件或配置数据。
 - UART：1 个 NS16550 兼容的 UART，用于串行通信。
 - RTC：1 个 Google Goldfish RTC，用于实时时钟功能。
 - VirtIO-MMIO 传输设备：8 个 VirtIO-MMIO 传输设备，用于虚拟化环境中的设备通信。
 - PCIe 主桥：1 个通用的 PCIe 主桥，用于连接 PCIe 设备。
 - fw_cfg 设备：允许 guest 从 QEMU 获取数据。

■ VIRT引导过程

- MROM: 0x0000_1000, Bootloader
- DRAM: 0x8000_0000, kernel

```
static const MemMapEntry virt_memmap[] = {  
    [VIRT_DEBUG] = { 0x0, 0x100 },  
    [VIRT_MROM] = { 0x1000, 0xf000 },  
    [VIRT_TEST] = { 0x100000, 0x1000 },  
    [VIRT_RTC] = { 0x101000, 0x1000 },  
    [VIRT_CLINT] = { 0x2000000, 0x10000 },  
    [VIRT_PCIE_PIO] = { 0x3000000, 0x10000 },  
    [VIRT_PLIC] = { 0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },  
    [VIRT_UART0] = { 0x10000000, 0x100 },  
    [VIRT_VIRTIO] = { 0x10001000, 0x1000 },  
    [VIRT_FLASH] = { 0x20000000, 0x4000000 },  
    [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },  
    [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },  
    [VIRT_DRAM] = { 0x80000000, 0x0 },  
};
```



■ 运行qemu

- `qemu-system-riscv32 -machine virt -bios none -nographic -serial mon:studio`
 - `-machine virt`: 启动一个 virt 机器。你可以用 `-machine '?'` 选项查看其他支持的机器。
 - `-bios none`: 不使用bios固件, `-bios default`: 使用默认固件 (如OpenSBI)
 - `-nographic`: 启动 QEMU 时不使用 GUI 窗口。
 - `-serial mon:stdio`: 将 QEMU 的标准输入/输出连接到虚拟机的串行端口。指定 `mon`: 允许通过按下 `Ctrl+A` 然后 `C` 切换到 QEMU 监视器。
 - `-kernel [要运行的elf文件]`
- 按 `Ctrl+A` 然后 `C` 切换到 QEMU 调试控制台

<code>C-a h</code>	打印此帮助
<code>C-a x</code>	退出模拟器
<code>C-a s</code>	将磁盘数据保存回文件 (如果使用 <code>-snapshot</code>)
<code>C-a t</code>	切换控制台时间戳
<code>C-a b</code>	发送中断 (<code>magic sysrq</code>)
<code>C-a c</code>	在控制台和监视器之间切换
<code>C-a C-a</code>	发送 <code>C-a</code>

■ 第一个程序

- 链接脚本 kernel.ld
 - 起始地址: 0x8000_0000
 - section: text、rodata、data、bss
 - 堆栈: 128K
 - 三个全局变量
 - `_bss_start`
 - `_bss_end`
 - `_stack_top`

```
OUTPUT_ARCH("riscv")
ENTRY("_start")
SECTIONS
{
    . = 0x80000000;
    .text : {
        |     *(.text .text.*)
    }
    .rodata : ALIGN(4){
        |     *(.rodata .rodata.*)
    }
    .data : ALIGN(4){
        |     *(.sdata .sdata.*)
        |     *(.data .data.*)
    }

    .bss : ALIGN(4){
        |     _bss_start = .;
        |     *(.sbss .sbss.*)
        |     *(.bss .bss.*)
        |     _bss_end = .;
    }

    . = ALIGN(4);
    . += 128 * 1024; /* 128K stack */
    _stack_top = . ;
}
```

■ 第一个程序

- 启动文件: start.s
- 只使用第一个hart来引导

```
1
2  .global _start
3
4  .text
5
6  _start:
7      csrr t0, mhartid      #读取hart id, 确保是0号
8      bnez t0, halt
9
10     la sp, _stack_top     #设置堆栈
11     j kernel_main         #进入main
12
13  halt:
14      wfi
15      j halt
```

■ 第一个程序

main.c

- 使用 extern char 获取链接器脚本符号。
只关心获取符号的地址，所以使用 char 类型或其他类型都可以。
- .bss段的初始化。
 - 首先使用 memset 函数将 .bss 段初始化为零。虽然某些引导加载程序可能会识别并清零 .bss 段，但我们以防万一手动初始化它

```
1  typedef unsigned char uint8_t;
2  typedef unsigned int uint32_t;
3  typedef uint32_t size_t;
4
5  void *memset(void *buf, char c, size_t n){
6      uint8_t *p = (uint8_t *)buf;
7
8      while(n--){
9          *p++ = c;
10     }
11     return buf;
12 }
13
14 extern char _bss_start[], _bss_end[];
15
16 void kernel_main(){
17
18     memset(_bss_start, 0, _bss_end - _bss_start);
19     while(1){}
20 }
21
```

■ 第一个程序

- Makefile

```
1  CROSS = riscv32-unknown-elf-
2  CC = $(CROSS)gcc
3  OBJDUMP = $(CROSS)objdump
4
5  CFLAGS = -std=c11 -g3 -Wall -march=rv32ima -mabi=ilp32 -nostdlib -fno-builtin -ffreestanding
6
7  QEMU = qemu-system-riscv32
8  QFLAGS = -nographic -machine virt -bios none -serial mon:stdio --no-reboot
9
10 LDFILE = kernel.ld
11 TARGET = kernel.elf
12 SRC = start.s kernel.c
13
14 all : $(TARGET)
15
16 $(TARGET) : $(SRC)
17 |     $(CC) $(CFLAGS) -T $(LDFILE) -o $@ $^
18 |
19 qemu : $(TARGET)
20 |     @echo "Press Ctl-A + c to QEMU monitor, Ctl-A + x to exit QEMU"
21 |     $(QEMU) $(QFLAGS) -kernel $(TARGET)
22 |
23 clean :
24 |     rm -f *.elf
```


■ 第一个程序

- 编译
 - make
- 运行
 - make qemu

```
xing@xinghz:~/chap4/4-5$ make qemu
```

```
Press Ctrl-A + c to QEMU monitor, Ctrl-A + x to exit QEMU
```

```
qemu-system-riscv32 -nographic -machine virt -bios none -serial mon:stdio --no-reboot -kernel kernel.elf
```

```
□
```

- 按Ctrl-A + c进入qemu控制台

■ 调试

- 按Ctrl-A + c进入qemu控制台
- 查看寄存器
 - info registers
- 暂停运行
 - stop
- 继续运行
 - cont
- 退出qemu
 - q

```
QEMU 8.2.2 monitor - type 'help' for more information
(qemu) █
```

■ 调试

- info registers

(qemu) info registers

CPU#0

V = 0

pc 800000a0

mhartid 00000000

mstatus 00000000

mstatush 00000000

hstatus 00000000

vsstatus 00000000

mip 00000080

mie 00000000

mideleg 00001444

hideleg 00000000

medeleg 00000000

hedeleg 00000000

mtvec 00000000

stvec 00000000

vstvec 00000000

mepc 00000000

sepc 00000000

vsepc 00000000

mcause 00000000

scause 00000000

vscause 00000000

mtval 00000000

stval 00000000

htval 00000000

mtval2 00000000

mscratch 00000000

sscratch 00000000

satp 00000000

x0/zero	00000000	x1/ra	800000a0	x2/sp	80020098	x3/gp	00000000
x4/tp	00000000	x5/t0	00000000	x6/t1	00000000	x7/t2	00000000
x8/s0	800200a8	x9/s1	00000000	x10/a0	800000a8	x11/a1	00000000
x12/a2	00000000	x13/a3	00000000	x14/a4	ffffffff	x15/a5	800000a8
x16/a6	00000000	x17/a7	00000000	x18/s2	00000000	x19/s3	00000000
x20/s4	00000000	x21/s5	00000000	x22/s6	00000000	x23/s7	00000000
x24/s8	00000000	x25/s9	00000000	x26/s10	00000000	x27/s11	00000000
x28/t3	00000000	x29/t4	00000000	x30/t5	00000000	x31/t6	00000000

■ 调试

- 反编译: `objdump -d kernel.elf`

```
80000000 <_start>:
80000000:      f14022f3          csrr    t0,mhartid
80000004:      00029863          bnez    t0,80000014 <halt>
80000008:      00020117          auipc   sp,0x20
8000000c:      0a010113          addi    sp,sp,160 # 800200a8 <_stack_top>
80000010:      06c0006f          j       8000007c <kernel_main>
```

```
80000014 <halt>:
80000014:      10500073          wfi
80000018:      ffdff06f          j       80000014 <halt>
```

```
8000007c <kernel_main>:
8000007c:      ff010113          addi    sp,sp,-16
80000080:      00112623          sw      ra,12(sp)
80000084:      00812423          sw      s0,8(sp)
80000088:      01010413          addi    s0,sp,16
8000008c:      00000613          li      a2,0
80000090:      00000593          li      a1,0
80000094:      800007b7          lui     a5,0x80000
80000098:      0a878513          addi    a0,a5,168 # 800000a8 <_stack_top+0xfffe0000>
8000009c:      f81ff0ef          jal     ra,8000001c <memset>
800000a0:      00000013          nop
800000a4:      ffdff06f          j       800000a0 <kernel_main+0x24>
```

■ qemu 日志

- qemu-system-riscv32 -nographic -machine virt -bios none -kernel kernel.elf

-d in_asm,cpu -D qemu.log

```
-----
IN:
Priv: 3; Virt: 0
0x00001000: 00000297      auipc      t0,0      # 0x1000
0x00001004: 02828613      addi      a2,t0,40
0x00001008: f1402573      csrrs     a0,mhartid,zero

V      =      0
pc      00001000
mhartid 00000000
mstatus 00000000
mstatush 00000000
hstatus 00000000
vsstatus 00000000
mip      00000080
mie      00000000
mideleg  00001444
hideleg  00000000
medeleg  00000000
hedeleg  00000000
mtvec    00000000
stvec    00000000
```

```
0x00001000: 00000297      auipc
0x00001004: 02828613      addi
0x00001008: f1402573      csrrs
0x0000100c: 0202a583      lw
0x00001010: 0182a283      lw
0x00001014: 00028067      jr
```

qemu从0x1000开始执行,
然后跳转到0x8000_0000



```
t0,0
a2,t0,40
a0,mhartid,zero
a1,32(t0)
t0,24(t0)
t0
```


■ 调试

- 查看符号/函数地址: `riscv32-unknown-elf-nm kernel.elf`

```
xing@xinghz:~/chap4/4-5$ riscv32-unknown-elf-nm kernel.elf
800000b0 T _bss_end
800000b0 T _bss_start
800200b0 T _stack_top
80000000 T boot
80000018 t halt
80000084 T kernel_main
80000024 T memset
```

■ 使用gdb调试

- 启动qemu:
 - `qemu-system-riscv32 -machine virt -nographic -bios none -kernel hello -S -s`
 - `-kernel hello`: 加载编译后的程序。
 - `-S`: 启动后暂停 CPU。
 - `-s`: 启动 GDB 服务器, 监听 1234 端口。

■ 使用内联汇编

- 可以用内联汇编将start.s代码放在kernel.c中
- boot 函数
- `__attribute__((naked))` 属性告诉编译器不要在函数体前后生成不必要的代码，比如返回指令。这确保内联汇编代码就是确切的函数体。
- `__attribute__((section(".text.boot")))` 属性，它控制函数在链接器脚本中的放置。将 boot 函数放在 0x80-00000

```
41 extern char _bss_start[], _bss_end[], _stack_top[];
42
43 void kernel_main(){
44
45     memset(_bss_start, 0, _bss_end - _bss_start);
46     while(1){}
47 }
48
49 __attribute__((section(".text.boot")))
50 __attribute__((naked))
51 void boot(void) {
52     __asm__ volatile(
53         "csrr t0, mhartid\n"
54         "bnez t0, halt\n"
55         "mv sp, %0\n" // 设置栈指针
56         "j kernel_main\n" // 跳转到内核主函数
57         "halt: wfi\n"
58         "j halt\n"
59         :
60         : "r" (_stack_top) // 将栈顶地址作为 %[stack_top] 传递
61         );
62 }
```

SECTIONS

```
{
    . = 0x80000000;
    .text : {
        KEEP(*(.text.boot));
        *(.text .text.*)
    }
}
```


■ `__attribute__((naked))`

```
1  extern char _stack_top[];
2
3  __attribute__((section(".text.boot")))
4  __attribute__((naked))
5  void boot(void) {
6      __asm__ volatile(
7          "csrr t0, mhartid\n"
8          "bnez t0, halt\n"
9          "mv sp, %0\n" // 设置栈指针
10         "j main\n"      // 跳转到内核主函数
11         "halt: wfi\n"
12         "j halt\n"
13         :
14         : "r" (_stack_top) // 将栈顶地址作为 :
15     );
16 }
17 int main(){
18
19
20     for(;;){}
21
22     return 0;
23 }
```



```
1  boot:
2      lui    a5,%hi(_stack_top)
3      addi   a5,a5,%lo(_stack_top)
4      csrr t0, mhartid
5  bnez t0, halt
6  mv sp, a5
7  j main
8  halt: wfi
9  ✓ j halt
10
11      nop
12
13  main:
14      addi   sp,sp,-16
15      sw     ra,12(sp)
16      sw     s0,8(sp)
17      addi   s0,sp,16
18
19  .L3:
20      j      .L3
```

■ OpenSBI

- `qemu -bios default`
 - `bios`
- OpenSBI (Open Supervisor Binary Interface) 是RISC-V架构中运行在机器模式 (M-Mode) 的固件, 实现了RISC-V SBI (Supervisor Binary Interface) 规范。它的核心作用是为运行在监管者模式 (S-Mode) 的操作系统 (如Linux) 提供统一的硬件抽象接口, 同时管理底层硬件的初始化与多核协作。
- OpenSBI预编译固件
 - https://github.com/qemu/qemu/raw/v8.0.4/pc-bios/opensbi-riscv32-generic-fw_dynamic.bin

■ OpenSBI

- 提供低级系统功能的抽象：
 - OpenSBI 为运行在监督器模式 (S-mode) 或虚拟监督器模式 (VS-mode) 的软件提供了对平台特定功能的抽象接口，例如 UART、定时器等。这使得操作系统内核可以使用统一的接口与硬件交互，而不需要关心具体的硬件实现细节。
- 支持多种平台：
 - OpenSBI 支持多种 RISC-V 平台，包括 SiFive 的 Freedom 平台、VisionFive 等。
 - 它可以运行在不同的硬件平台上，提供一致的系统服务。
- 增强系统启动过程：
 - OpenSBI 是系统启动过程中的第一个运行的组件，它负责初始化硬件并加载引导程序或操作系统。解析设备树 (Device Tree)，帮助操作系统识别和配置硬件设备。
- 提供安全访问：
 - OpenSBI 运行在机器模式 (M-mode)，具有对硬件的完全访问权限。
 - 它为监督器模式的软件提供了安全的接口，防止直接访问硬件导致的安全问题。

■ OpenSBI

- OpenSBI 的应用场景
- 嵌入式系统开发：
 - 在嵌入式系统中，OpenSBI 可以帮助开发者快速启动和调试系统，提供对硬件的低级访问。
- 操作系统移植：
 - 在将操作系统（如 Linux）移植到 RISC-V 平台时，OpenSBI 提供了必要的系统服务，简化了移植过程。
- 硬件验证：
 - OpenSBI 可以用于验证 RISC-V 硬件平台的功能和性能，确保硬件符合 RISC-V 规范。

■ 串口输出: hello world

- NS16550
 - 基地址: 0x1000_0000

```
static const MemMapEntry virt_memmap[] = {
    [VIRT_DEBUG] = { 0x0, 0x100 },
    [VIRT_MROM] = { 0x1000, 0xf000 },
    [VIRT_TEST] = { 0x100000, 0x1000 },
    [VIRT_RTC] = { 0x101000, 0x1000 },
    [VIRT_CLINT] = { 0x2000000, 0x10000 },
    [VIRT_PCIE_PIO] = { 0x3000000, 0x10000 },
    [VIRT_PLIC] = { 0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },
    [VIRT_UART0] = { 0x10000000, 0x100 },
    [VIRT_VIRTIO] = { 0x10001000, 0x1000 },
    [VIRT_FLASH] = { 0x20000000, 0x4000000 },
    [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },
    [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },
    [VIRT_DRAM] = { 0x80000000, 0x0 },
};
```

■ ns16550

- RBR (0: 读) 、THR (0: 写) 接收和发送寄存器
- LSR (5: 读) 线路状态寄存器
- LCR (3: 写) 线路控制寄存器
- IER (1: 写) 中断使能寄存器
- ISR (2: 读) 中断状态寄存器

A2	A1	A0	READ MODE	WRITE MODE
0	0	0	• <u>Receive Holding Register</u>	• <u>Transmit Holding Register</u>
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• <u>Interrupt Enable Register</u>
0	0	1	N/A	• MSB of Divisor Latch when Enabled
0	1	0	• <u>Interrupt Status Register</u>	• FIFO control Register
0	1	1	N/A	• <u>Line Control Register</u>
1	0	0	N/A	• Modem Control Register
1	0	1	• <u>Line Status Register</u>	N/A
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read	• Scratchpad Register Write

■ LSR：线路状态寄存器

LSR (5：读) 线路状态

位	含义	描述
Bit 7	FIFO Error	表示 FIFO 错误，当 FIFO 发生错误时，该位被置 1。
Bit 6	Transmit Empty	表示发送寄存器为空，当发送保持寄存器（THR）和发送移位寄存器（TSR）都为空时，该位被置 1。
Bit 5	Transmit Holding Empty	表示发送保持寄存器为空，当 THR 为空时，该位被置 1。
Bit 4	Break Interrupt	表示断开中断，当接收到一个断开条件时，该位被置 1。
Bit 3	Framing Error	表示帧错误，当接收到的数据帧格式错误时，该位被置 1。
Bit 2	Parity Error	表示奇偶校验错误，当接收到的数据奇偶校验错误时，该位被置 1。
Bit 1	Overrun Error	表示溢出错误，当接收到新的字符但接收保持寄存器（RHR）读取时，该位被置 1。
Bit 0	Receive Data Ready	表示接收数据就绪，当接收移位寄存器（RSR）中有数据可读时，该位被置 1。

■ 线路控制寄存器LCR

- LCR (3: 写) 线路控制寄存器
 - 波特率
 - 奇偶校验位
 - 停止位长度
 - 数据我长度

位	含义	描述
Bit 7	Divisor Latch Access Bit (DLAB)	当该位为 1 时，允许访问波特率除数寄存器为 0 时，访问的是 THR 和 RHR 寄存器。
Bit 6	Set Break	当该位为 1 时，会在传输线上强制产生一个将串行输出引脚 (TXD) 强制到空闲状态 (
Bit 5	Set Parity	当该位为 1 时，启用奇偶校验功能。
Bit 4	Even Parity	当该位为 1 时，选择偶校验；当该位为 0 时
Bit 3	Parity Enable	当该位为 1 时，启用奇偶校验功能。
Bit 2	Stop Bits	该位决定了数据传输结束时的停止位数量：2 个停止位 (对于 5 位数据长度，表示 1.5
Bit 1 和 Bit 0	Word Length	这两位决定了每个字符的数据位数量：00 表示 7 位，11 表示 8 位。

■ 中断相关寄存器IER和ISR

IER (1: 写) 中断使能寄存器

ISR (2: 读) 中断状态寄存器

- 000 : 保留
- 001 : 发送保持寄存器为空中断
- 010 : 接收数据就绪中断
- 011 : 接收线路状态中断
- 100 : 调制解调器状态中断
- 101 : 保留
- 110 : 字符超时中断
- 111 : 保留 |

位	含义	描述
Bit 0	Data Ready Interrupt Enable	使能接收数据就绪中断
Bit 1	Transmitter Holding Register Empty Interrupt Enable	使能发送保持寄存器为空中断
Bit 2	Receive Line Status Interrupt Enable	使能接收线路状态中断
Bit 3	Modem Status Interrupt Enable	使能调制解调器状态中断

位	含义	描述
Bit 0	Interrupt Pending	中断挂起标志, 0 表示有中断挂起, 1 表示无中断挂起
Bit 1-3	Interrupt Identifier	中断类型标识:

■ 串口输出字符

- putc
- puts
- ...
- printf

```
#define IOB(addr) (*(volatile char *)(addr))
#define IOW(addr) (*(volatile int *)(addr))
```

```
#define UART 0x10000000
#define UART_THR (UART + 0) //发送数据寄存器, THR:transmitter holding register
#define UART_LSR (UART + 0x05) //状态寄存器, LSR:line status register
#define UART_LSR_EMPTY 0x40 //LSB bit 6,发送缓冲区空
```

```
int lib_putc(char c){
    while( IOB(UART_LSR) & UART_LSR_EMPTY == 0) ;

    IOB(UART_THR) = c;

    return c;
}
```

```
int lib_puts(char *s){
    while(*s){
        lib_putc(*s++);
    }
}
```

■ 串口输出数字、printf

```
int lib_putd(int d){
    if(d < 0){
        lib_putc('-');
        d = -d;
    }

    if(d < 10){
        lib_putc(d + '0');
    } else {
        lib_putd( d / 10);
        lib_putc(d % 10 + '0');
    }
}

void lib_puth(int x){
    for(int i = 7;i >= 0;i--){
        lib_putc("0123456789abcdef"[ (x >> (i*4)) & 0xf]);
    }
}
```

■ 打印hello world

- main

```
void kernel_main(){  
void kernel_main(){  
    ;  
    memset(_bss_start, 0, _bss_end - _bss_end);  
    panic("启动...panic");  
    lib_printf("不会到此");  
}
```

```
> xing@xinghz:~/chap4/4-5$ make qemu  
Press Ctl-A + c to QEMU monitor, Ctl-A + x to exit QEMU  
qemu-system-riscv32 -nographic -machine virt -bios none -serial mon:stdio --no-reboot -kernel kernel.e  
lf  
hello world
```

■ panic

panic: 当遇到不可恢复的错误时，打印出错语句所在的的文件和行号，并停止运行。

文件名: `__FILE__`，行号: `__LINE__`

```
#define panic(fmt, ...) \
do { \
    lib_printf("PANIC: %s:%d: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__ ); \
    while(1) {} \
}while(0)
```

do-while 语句。由于它是 while (0)，这个循环只执行一次。这是定义由多个语句组成的宏的常见方式。简单地用 { ... } 封装可能会在与 if 等语句组合时导致意外的行为。另外，注意每行末尾的反斜杠 (\)。虽然宏是在多行上定义的，但在展开时换行符会被忽略。

`## __VA_ARGS__`。这是一个用于定义接受可变数量参数的宏的有用编译器扩展（参考：GCC 文档）。当可变参数为空时，`##` 会删除前面的 `,`。这使得即使只有一个参数，如 `PANIC("booted!")`，编译也能成功。

■ panic

- main

```
void kernel_main(){  
    memset(_bss_start, 0, _bss_end - _bss_start);  
    panic("启动...panic");  
    lib_printf("不会到此");  
}
```

```
qemu-system-riscv32 -nographic -machine v  
PANIC: kernel.c:113: 启动...panic
```


■ printf

- main

```
void kernel_main(){  
    memset(_bss_start, 0, _bss_end - _bss_start);  
    lib_puts("hello world\n");  
    while(1){}
```

```
> xing@xinghz:~/chap4/4-5$ make qemu  
Press Ctl-A + c to QEMU monitor, Ctl-A + x to exit QEMU  
qemu-system-riscv32 -nographic -machine virt -bios none -serial mon:stdio --no-reboot -kernel kernel.e  
lf  
hello world
```

■ 特权级别

RISC-V指令集定义了三个特权级别：

- U: 00, 用户模式;
 - S: 01, 监督模式;
 - M: 11, 机器模式
-
- V: 虚拟机模式

机器模式具有最高权限，允许完全访问硬件。监督模式具有第二高权限，而用户模式具有最少权限。

■ 特权级别

RISC-V硬件平台可以实现一个子集或所有这些特权级别。

例如，当为紧凑而简单的嵌入式系统实现硬件平台时，可能只需要机器特权级别。

当依赖于**操作系统**来管理应用程序的时，通常需要包含所有三个特权级别以实现操作系统。

RISC-V定义了当前正在执行的特权模式软件的特权级别。当机器特权模式处于活动状态时，当前正在执行的软件具有机器特权级别，因此可以完全访问硬件。这是具有最高特权的特权模式。

在RISC非特权模式V中，非特权模式是用户/应用程序特权模式，也称为用户模式。这是在非特权模式下运行的软件可以访问的指令集的子集。

为了简化讨论，将重点关注只有两种特权模式的RISC-V处理器：**用户模式**和**机器模式**。

■ 保护系统

用户/应用程序模式限制了当前执行软件可以访问的资源。为了保护系统免受错误或恶意用户程序的影响，系统软件通常在执行（或将控制权返回）用户代码之前将权限模式设置为用户模式。

通常采取以下操作来保护系统：

- **配置系统**：开机时，硬件自动将权限模式设置为机器模式并开始执行引导代码。引导代码将操作系统软件加载到内存中，并在机器模式下调用其初始化代码，这允许操作系统配置整个系统。
- **执行用户代码**：一旦设置了系统，操作系统就可以将用户程序加载到主存储器中并执行它们。在执行用户代码之前，它将特权模式设置为用户模式。
- **处理非法操作**：如果用户软件试图执行特权操作，例如与外围设备交互，硬件停止执行用户代码并调用操作系统，以便它可以处理非法操作。硬件使用**异常处理机制**将控制权转移给操作系统。

■ 保护系统

- **调用操作系统：**如果用户程序需要执行敏感的过程，例如输出到外设，它必须调用操作系统，操作系统将代表用户程序执行该过程。当将控制权转移到操作系统时，硬件必须将特权模式更改为监管模式或机器模式，以便操作系统可以以适当的特权执行。
- ISA通常包括一种称为软件中断的机制，该机制允许在非特权模式下运行的代码调用系统代码并同时更改特权模式。这种机制的设计使得用户代码不能更改特权模式并执行自己的代码。
- 一旦操作系统完成代表用户程序执行该过程，它会将特权模式更改回用户/应用程序模式并返回用户程序。
- **处理外部中断：**在外部中断时，硬件将特权模式设置为机器模式，因此中断服务例程有足够的特权来处理中断。中断服务例程属于系统软件。

■ 更改权限模式

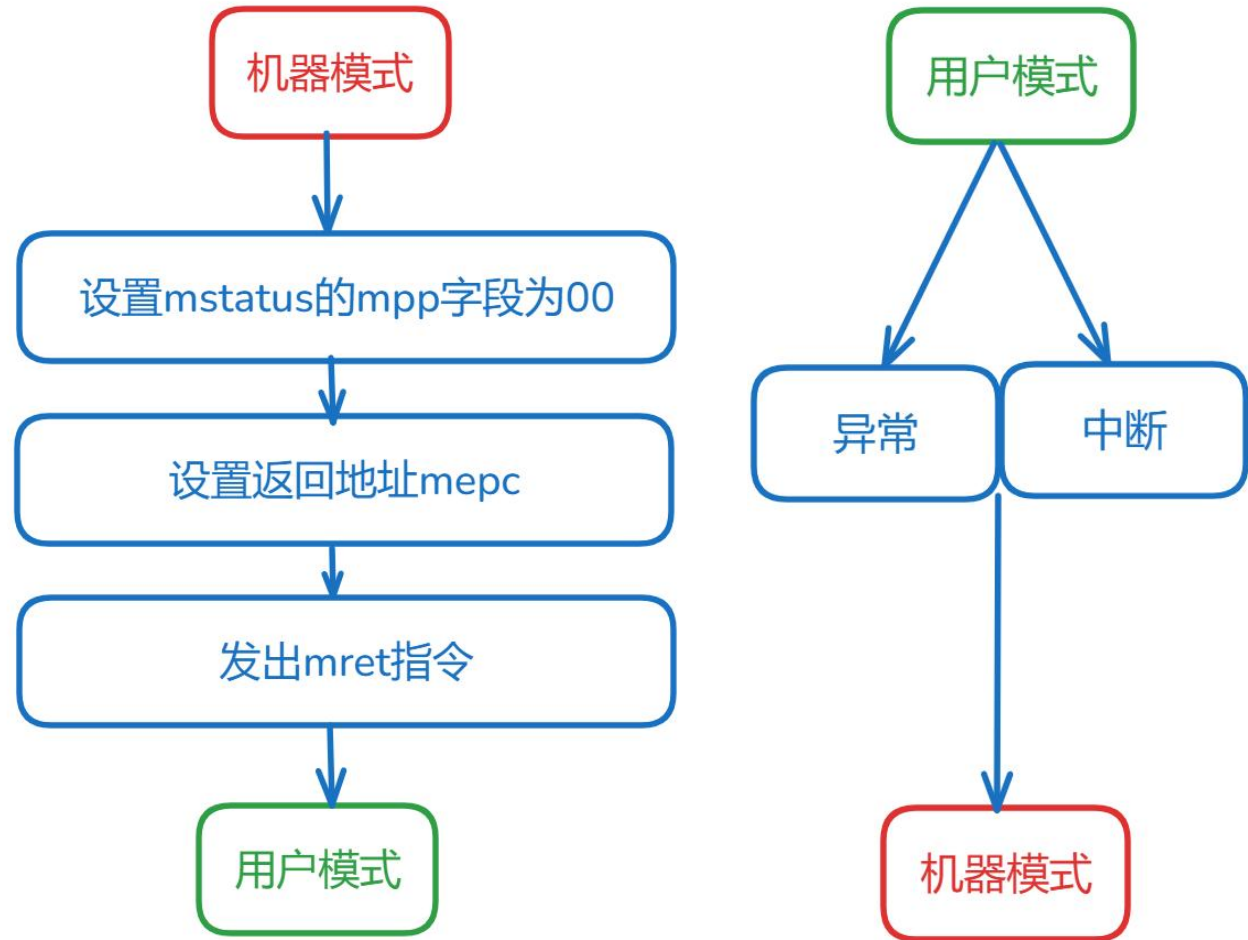
RISC-V CPU将当前特权模式存储在用户不可见的内部CSR中。

用户软件无法直接检查或修改当前特权模式。

检查当前特权模式的唯一方法是生成软件中断或异常，将特权模式代码复制到 **mstatus.MPP** 字段 (机器先前特权字段)。

设置当前特权模式的唯一方法是修改 **mstatus.MPP** 字段的内容，并执行 **mret** 指令。该指令使用当前mstatus MPP字段中的值来设置当前特权模式。

注意mret返回的代码要有访问权限，否则会产生code access fault 异常



■ 更改权限模式

以下代码显示了系统软件如何将特权模式更改为用户/应用程序模式并同时调用用户程序。

首先，它将 **mstatus MPP** 字段设置为 “00”，即用户/应用程序模式。然后，将用户程序入口点加载到 **mepc** 中。最后，执行 **mret** 指令，该指令使用 **mstatus MPP** 的值更改特权模式，并同时使用 **mepc** 的值来设置程序计数器 PC。

```

1  # Changing to User/Application mode
2  csrr t1, mstatus          # Update the mstatus.MPP
3  li t2, ~0x1800           # field (bits 11 and 12)
4  and t1, t1, t2           # with value 00 (U-mode)
5  csrw mstatus, t1
6
7  la t0, user_main         # Loads the user software
8  csrw mepc, t0            # entry point into mepc
9
10 mret                     # PC <= MEPC; mode <= MPP;

```


■ 内存保护PMP

PMP (Physical Memory Protection) 是 RISC-V 架构中的一种硬件机制，用于控制不同内存区域的读、写和执行权限，主要提供对物理内存的保护和隔离。

PMP 的使用是通过设置一对 PMP 寄存器操作的，包括：

pmpaddr：用于设置受保护的地址区域。

pmpcfg：用于设置受保护区域的读、写、执行权限以及地址匹配模式。

每一对 PMP 寄存器称为一个 PMP 条目 (entry)，支持的条目数量和不同 CPU 硬件实现相关，通常条目数量是 8、16、32 和 64 不等。

```
li t0, 0xFFFFFFFF    # NAPOT编码：最低32位全1
csrw pmpaddr0, t0      # 区域大小=2^(32) = 4GB

li t1, 0x1F            # 二进制00011111 (A=3, R=1, W=1, X=1, L=0)
csrw pmpcfg0, t1       # 写入PMPCFG0

csrw pmpcfg1, 0        # 禁用PMP8-15 (若存在)
```

■ 异常

异常是CPU在执行指令时响应异常条件而生成的事件。例如，尝试执行非法指令是导致RISC-V CPU生成异常的条件。

异常通常会触发异常处理机制，以便在CPU继续执行程序之前处理异常条件。

这种机制通常会导致CPU将执行流程重定向到异常处理例程，该例程可能：

- 保存当前程序上下文；
- 处理异常条件；
- 恢复保存程序的上下文以继续执行

■ 异常

异常处理流程与用于处理硬件中断的流程非常相似。

RISC-V CPU使用相同的机制来处理**中断**和**异常**，它保存部分当前上下文（例如PC内容），设置CSR，并将执行流程重定向到中断服务例程。

中断服务例程可以通过检查中断原因寄存器mcause和状态寄存器mstatus，来区分中断和异常。

- mcause INTERRUPT（最高位）表示中断还是异常
- mcause.EXCCODE（低31位）表示中断或异常的来源。

■ 异常

RISC-V定义了多种异常和中断源。下表显示了中断和异常的来源及其相应编码。

mcause fields		Cause
INTERRUPT	EXCCODE	
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	Reserved
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	Reserved for future standard use
0	15	Store/AMO page fault
0	16-23	Reserved for future standard use
0	24-31	Reserved for custom use
0	32-47	Reserved for future standard use
0	48-63	Reserved for custom use
0	≥ 64	Reserved for future standard use

mcause fields		Cause
INTERRUPT	EXCCODE	
1	0	User software interrupt
1	1	Supervisor software interrupt
1	2	Reserved for future standard use
1	3	Machine software interrupt
1	4	User timer interrupt
1	5	Supervisor timer interrupt
1	6	Reserved for future standard use
1	7	Machine timer interrupt
1	8	User external interrupt
1	9	Supervisor external interrupt
1	10	Reserved for future standard use
1	11	Machine external interrupt
1	12-15	Reserved for future standard use
1	≥ 16	Reserved for platform use

■ 异常

异常处理机制通常用于保护系统免受非法用户代码操作。

在这种情况下，硬件由系统软件配置，以在特权模式设置为用户/应用程序并且CPU尝试执行某些操作时生成异常，例如访问映射到外围设备的地址或访问只能在机器模式下访问的控制和状态寄存器。

在异常时，属于系统软件的中断服务例程可以决定如何处理用户程序。

■ 配置异常和中断机制

RISC-V CPU使用类似的机制来处理中断、异常和软件中断，它保存了部分当前上下文（PC内容），设置了例如mcause和其他CSR，并将执行流程重定向到中断服务例程。因此，配置异常和软件中断处理机制与配置外部中断处理机制非常相似。

系统软件通过注册将处理这些事件的例程来配置异常和软件中断机制。这与注册中断服务例程来处理外部中断的方式相同。

在直接模式下，注册一个异常处理例程，负责检查mcause，识别事件源并调用正确的处理函数。在向量模式下，外部中断、异常和软件中断处理例程必须在中断向量表中注册。

在RISC-V上，必须通过设置控制和状态寄存器的mstatus来启用外部中断。另一方面，异常和软件中断总是启用的，不需要额外的配置。

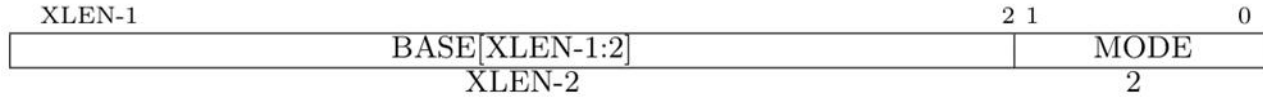
■ 配置异常和中断机制

- 在 RISC-V 中，异常将按以下方式处理：
 - CPU 检查 medeleg 寄存器以确定哪个操作模式应该处理异常。默认是机器模式下处理
 - CPU 将其状态（寄存器）保存到各种 CSR 中。
 - mcause、mtval、mepc
 - scause、stval、sepc
 - ucause、utval、uepc
 - mtvec/stvec/utvec 寄存器的值被设置为程序计数器PC，跳转到内核的异常处理程序。
 - 异常处理程序保存通用寄存器（即程序状态），并处理异常。
 - 完成后，异常处理程序恢复保存的执行状态并调用 mret/sret/uret 指令，从发生异常的地方恢复执行。

■ 异常处理程序

- start.s

- 设置异常处理入口地址: `_trap_entry`
- 直接模式和向量模式
 - `mtvec`最低两位, 00直接模式, 01向量模式
- **直接模式**: 只需要设置一个统一的异常处理入口地址。所有中断和异常共用一个入口: 所有的中断和异常都跳转到同一个地址, 由软件在处理函数中通过读取 `mcause` 寄存器来判断具体的异常或中断类型 (BASE)



```
_start:
    csrr t0, mhartid    #读取hart id, 确保是0号
    bnez t0, halt

    la sp, _stack_top   #设置堆栈

    la a0, _trap_entry  #设置异常处理
    csrw mtvec, a0

    call kernel_main    #进入main
```

■ 异常处理程序

- **向量模式**：每个中断号对应一个独立的处理函数，处理器会根据中断号直接跳转到对应的处理函数。需要构建一个异常向量表，每个表项是对应的中断处理函数
- 处理程序入口地址为：BASE+4 * 中断号
- 所有的异常处理入口：BASE。
- 向量模式下，异常和0号中断（user software interrupt）处理程序入口地址都是BASE（第一项），通过检查mcause来区分。

```
la    t0, vector_table    # 加载中断向量表的基地址

li    t1, 1                # 设置 MODE = 1 (向量模式)
or    t0, t0, t1           # 组合 BASE 和 MODE

csrw  mtvec, t0            # 写入 mtvec
```

```
.align 2
vector_table:
    j handler_0    # 中断 ID 0
    j handler_1    # 中断 ID 1
    j handler_2    # 中断 ID 2
```

```
handler_0:
    # 处理中断 0 的逻辑
    mret
```

```
handler_1:
    # 处理中断 1 的逻辑
    mret
```

■ 异常处理程序_trap_entry

- _trap_entry
 - 保护现场
 - 处理异常
 - 恢复现场
 - mret/sret/uret (返回地址在epc中设置)
- 要保护什么?
 - 中断和异常发生时, 当前处理器状态 (主要是寄存器)
 - 将保护的内容放在堆栈中
 - 退出前从堆栈恢复
- 处理异常
 - 根据mcause, 分情况处理

■ 异常处理程序_trap_entry

- _trap_entry

- 1、保护现场 (31个寄存器x1-x31)

```
csrw mscratch, sp
```

先用mscratch保存当前堆栈指针sp

```
addi sp, sp, -4 * 31
```

堆栈分配31个寄存器存储空间

```
sw x1, 4 * 0(sp)
```

```
#sp 4 * 1(sp)
```

```
sw x3, 4 * 2(sp)
```

```
sw x4, 4 * 3(sp)
```

```
sw x5, 4 * 4(sp)
```

```
sw x6, 4 * 5(sp)
```

```
...
```

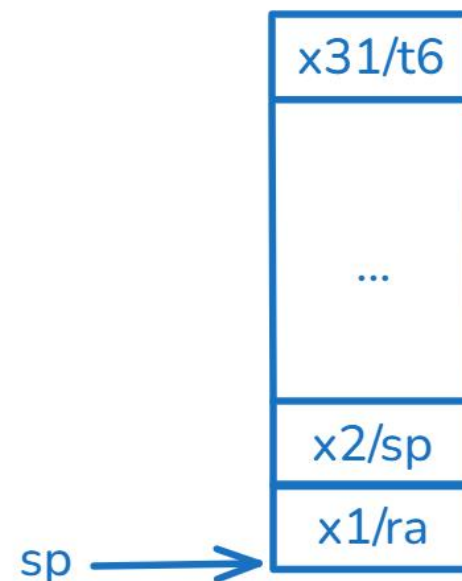
```
sw x30, 4 * 29(sp)
```

```
sw x31, 4 * 30(sp)
```

```
csrr a0, mscratch
```

```
sw a0, 4 * 1(sp)
```

保存老的sp到堆栈



■ 异常处理程序_trap_entry

- _trap_entry

- 2、异常处理

```
mv a0, sp
call handle_trap
```

a0指向栈顶，作为参数传给
handle_trap

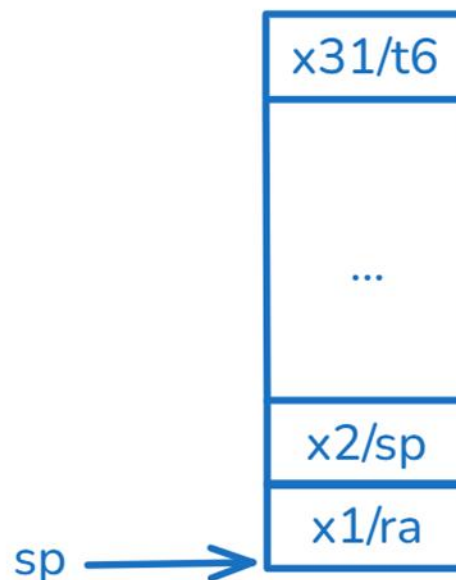
void handle_trap(struct regfile *)

- 3、恢复现场，并返回

```
lw x1, 4 * 0(sp)
lw x3, 4 * 2(sp)
lw x4, 4 * 3(sp)
...
lw x30, 4 * 29(sp)
lw x31, 4 * 30(sp)
```

返回

```
lw sp, 4 * 1(sp)
mret
```



```
typedef uint32_t reg_t;

struct regfile {
    reg_t ra;
    reg_t sp;
    reg_t gp;
    reg_t tp;
    reg_t t0;
    ...
    reg_t t5;
    reg_t t6;
} __attribute__((packed));
```

■ 异常处理：示例1

```
void handle_trap(struct regfile *rs){
    uint32_t cause, val, epc;

    CSRR(mcause, cause);
    CSRR(mtval, val);
    CSRR(mepc, epc);

    panic("unexpected trap cause=%x, tval=%x, epc=%x\n", cause, val, epc);
}
```

```
void kernel_main(){

    lib_printf("hello kernel\n");
    lib_printf("hello world\n");

    __asm__ volatile("unimp");

    panic("never come here\n");
}
```

unimp 是一个伪指令。汇编器将 unimp 转换为以下指令：
csrrw x0, cycle, x0
这会将 cycle 寄存器读取并写入 x0。由于 cycle 是只读寄存器，CPU 判断该指令无效并触发非法指令异常。



■ 异常处理：示例2

```
void handle_trap(struct regfile *rs){
    uint32_t cause, val, epc;

    CSRR(mcause, cause);
    CSRR(mtval, val);
    CSRR(mepc, epc);

    lib_printf("unexpected trap cause=%x, tval=%x, epc=%x\n", cause, val, epc);

    if(cause < 0x80000000){
        epc = epc + 4;
        CSRW(mepc, epc);
    }
}
```

```
void kernel_main(){

    lib_printf("hello kernel\n");
    lib_printf("hello world\n");
    int i = 0;
    for(;;){
        asm volatile("unimp");

        lib_printf("resumed from exception: %d\n", i++);
        lib_delay(1000);
    }
}
```

mcause最高位：1表示中断，0异常

异常发生时，mepc存放的是引起异常的指令地址（pc）

中断发生时，mepc存放的是下一条指令地址（pc+4）

■ 异常处理：示例3

- 系统调用
 - ecall指令会触发系统调用异常
 - 用户模式下ecall异常为8，监督模式下为9，机器模式下为11
 - 通常通过ecall为用户提供访问底层硬件接口

■ 异常处理：示例3

- 系统调用

- 用户模式下ecall异常为8，监督模式下为9，机器模式下为11

```
void handle_trap(struct regfile *rs){
    uint32_t cause, val, epc;

    CSRR(mcause, cause);
    CSRR(mtval, val);
    CSRR(mepc, epc);

    if(cause < 0x80000000){
        if(cause == 8 || cause == 9 || cause == 11) //ecall
            handle_syscall(rs);
        else
            lib_printf("unexpected exception cause=%x, tval=%x, epc=%x\n", cause, val, epc);
        epc = epc + 4;
        CSRW(mepc, epc);
    }
}
```

■ 异常处理：示例3

- 系统调用
 - 用户模式下ecall异常为8，监督模式下为9，机器模式下为11
 - ecall：假定a3为功能号，a0-a2为参数

```
#define SYS_PUTC 1
void handle_syscall(struct regfile *f) {
    switch (f->a3) {
        case SYS_PUTC:
            lib_putc(f->a0);
            break;
        default:
            PANIC("unexpected syscall a3=%x\n", f->a3);
    }
}
```

■ 异常处理：示例3

- 系统调用

```
int syscall(int sysno, int arg0, int arg1, int arg2) {  
    register int a0 __asm__("a0") = arg0;  
    register int a1 __asm__("a1") = arg1;  
    register int a2 __asm__("a2") = arg2;  
    register int a3 __asm__("a3") = sysno;  
  
    asm volatile("ecall"  
                 : "=r"(a0)  
                 : "r"(a0), "r"(a1), "r"(a2), "r"(a3)  
                 : "memory");  
  
    return a0;  
}  
  
void putchar(char ch) {  
    syscall(SYS_PUTC, ch, 0, 0);  
}
```

■ 中断

中断处理流程和异常是类似的。

三类中断：software interrupt、timer、external

几个中断相关寄存器：

mie：中断使能寄存器、mstatus：全局中断允许

进入中断后，全局中断被关闭（硬件）

离开中断前，全局中断恢复为原有状态（硬件）

1	0	User software interrupt
1	1	Supervisor software interrupt
1	2	Reserved for future standard use
1	3	Machine software interrupt
1	4	User timer interrupt
1	5	Supervisor timer interrupt
1	6	Reserved for future standard use
1	7	Machine timer interrupt
1	8	User external interrupt
1	9	Supervisor external interrupt
1	10	Reserved for future standard use
1	11	Machine external interrupt
1	12-15	Reserved for future standard use
1	≥ 16	Reserved for platform use

```
void handle_trap(struct regfile *f){
    uint32_t cause, val, epc;

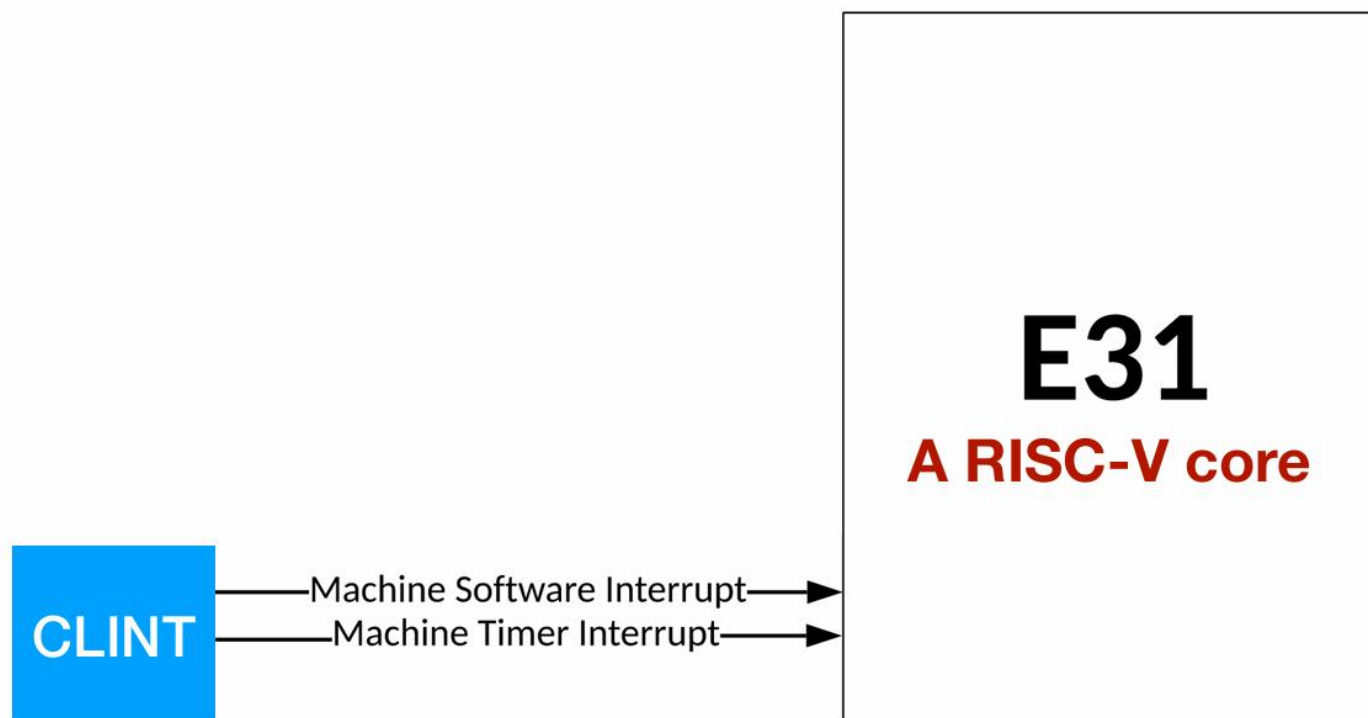
    CSRR(mcause, cause);
    CSRR(mtval, val);
    CSRR(mepc, epc);

    if(cause < 0x80000000){
        //异常处理
        epc = epc + 4;
        CSRW(mepc, epc);
    } else { //中断处理
        cause = cause & 0x7fffffff;
        switch(cause){

        }
    }
}
```

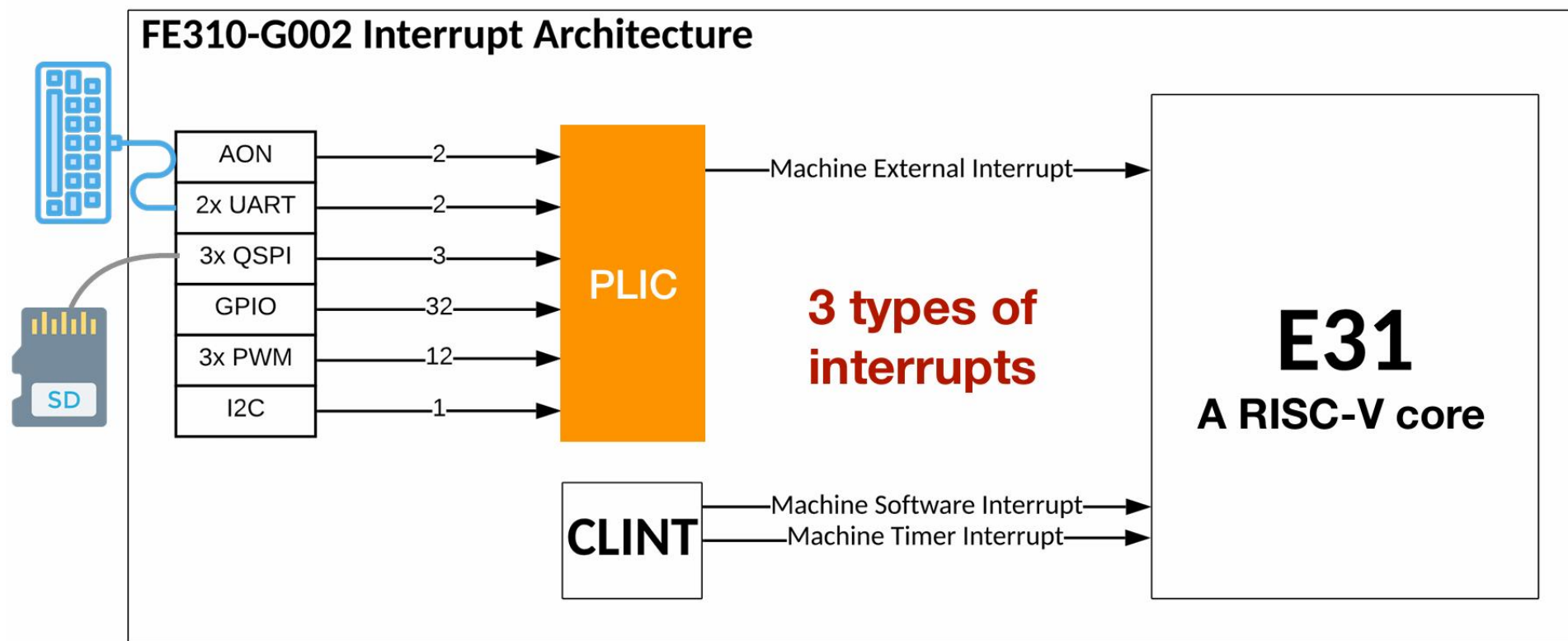
■ Core-local Interrupt (CLINT)

CLINT控制器向core发出software和timer中断
外部中断由PLINT负责统一处理



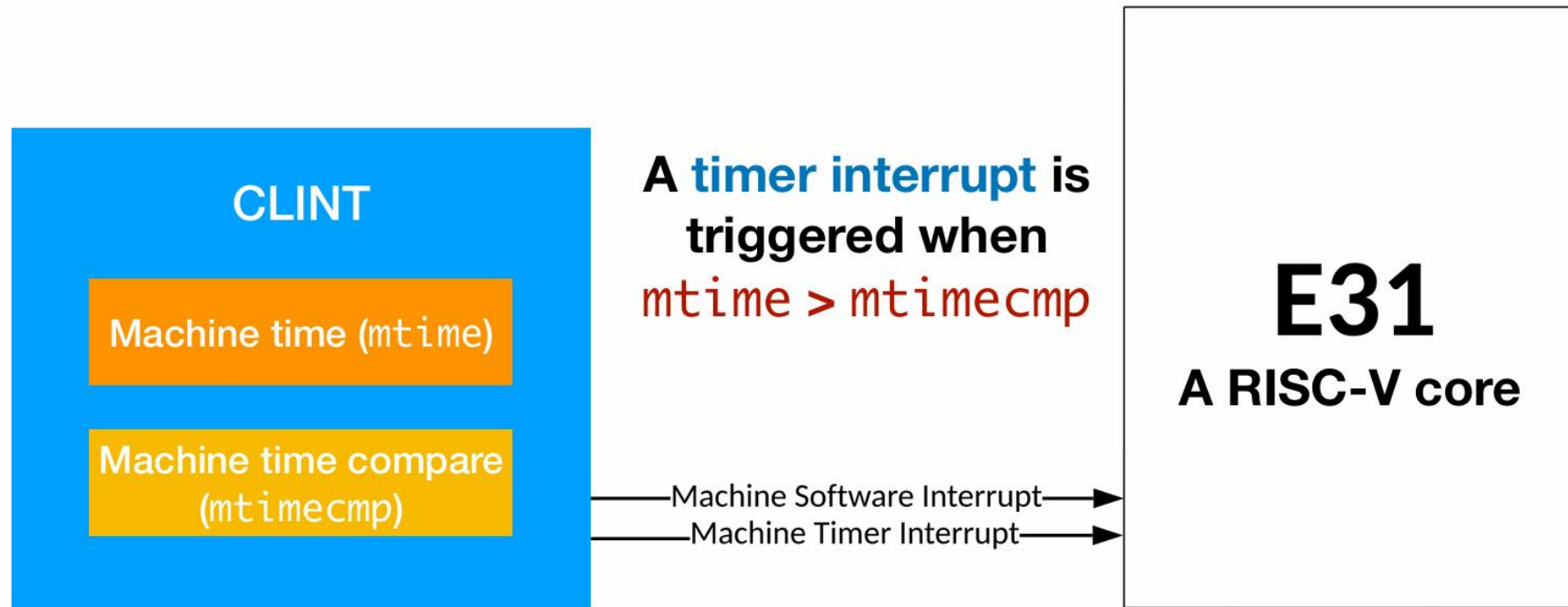
■ Platform Interrupt Controller (PLIC)

CLINT控制器向core发出software和timer中断
外部中断由PLIC负责统一处理



■ 定时器

- mtime: 当前时钟寄存器, 64位
- mtimecmp: 比较寄存器, 64位
- 当 $\text{mtime} > \text{mtimecmp}$ 会触发定时器中断



■ 定时器

- 定时器初始化
 - $\text{mtimecmp} = \text{mtime} + \text{定时间隔}$
- 定时器中断处理里面后再次设置
 - $\text{mtimecmp} = \text{mtime} + \text{定时间隔}$

```
int quantum = 50000;

void handler() {
    . . .
    mtimecmp_set(mtime_get() + quantum);
}

int main() {
    /* Set a timer */
    mtimecmp_set(mtime_get() + quantum);
    . . .
}
```


■ 定时器

- CLINT的CSR映射到存储空间

- `uint64_t current_mtime = *(volatile uint64_t*)CLINT_MTIME;`
- `uint64_t interval = 1000000;`
- `*(volatile uint64_t*)CLINT_MTIMECMP(hartid) = current_mtime + interval;`

`mtimecmp_set()`
writes 8 bytes to

`mtime_get()`
reads 8 bytes from

Address	Width	Attr.	Description
0x20000000	4B	RW	msip for hart 0
0x2004008			Reserved
...			
0x200bfff7			
<u>0x2004000</u>			
0x2004008	8B	RW	mtimecmp for hart 0
...			
0x200bfff7			
<u>0x200bfff8</u>			
0x200c000			Reserved

■ 允许定时器中断

mstatus: MIE, 机器中断允许

31		30								23		22	21	20	19	18	17		
SD		WPRI										TSR	TW	TVM	MXR	SUM	MPRV		
1		8										1	1	1	1	1	1		
16		15	14		13	12	11	10	9	8	7	6	5	4	3	2	1	0	
XS[1:0]		FS[1:0]		MPP[1:0]		WPRI	SPP	MPIE	WPRI	SPIE	UPIE	MIE	WPRI	SIE	UIE				
2		2		2		2	1	1	1	1	1	1	1	1	1				

```
asm("csrr %0, mstatus" : "=r"(mstatus));  
asm("csrw mstatus, %0" :: "r"(mstatus | 0x8));
```

■ 允许定时器中断

mie: MTIE, 机器定时器中断允许

XLEN-1	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI	MEIE	WPRI	SEIE	UEIE	MTIE	WPRI	STIE	UTIE	MSIE	WPRI	SSIE	USIE	
XLEN-12	1	1	1	1	1	1	1	1	1	1	1	1	1

```
asm("csrr %0, mie" : "=r"(mie));  
asm("csrw mie, %0" :: "r"(mie | 0x80));
```

■ mtime_get、mtimecmp_set

```
#define MTIMECMP 0x02004000
#define MTIME 0x0200bff8
void mtimecmp_set(long long t){
    IOW(MTIMECMP+4) = 0xffffffff;
    IOW(MTIMECMP) = (int)t;
    IOW(MTIMECMP+4) = (int)(t>>32);
}

long long mtime_get(){
    int low, high;
    do{
        high = IOW(MTIME+4);
        low = IOW(MTIME);
    }while(IOW(MTIME+4) != high);

    return (((long long)high) << 32) | low;
}
```

■ 定时器初始化、定时器中断处理

。

```
#define TIMER_INTERVAL 10000000 //1秒
```

```
void timer_init(){  
    mtimecmp_set(mtime_get() + TIMER_INTERVAL);  
  
    int v ;  
    CSRR(mie, v);  
    CSRW(mie, v | 0x80);  
  
    CSRR(mstatus, v);  
    CSRW(mstatus, v | 0x8);  
}
```

■ 定时器中断处理

```
void timer_handler(struct regfile *f){
    static int tick = 0;

    mtimecmp_set(mtime_get() + TIMER_INTERVAL);

    lib_printf("timer interrupt: %d\n", ++tick);
}
```

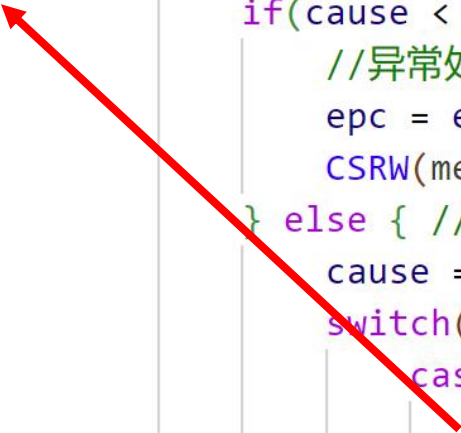
```
void kernel_main(){
    timer_init();

    for(;;){}
```

```
void handle_trap(struct regfile *f){
    uint32_t cause, val, epc;

    CSRR(mcause, cause);
    CSRR(mtval, val);
    CSRR(mepc, epc);

    if(cause < 0x80000000){
        //异常处理
        epc = epc + 4;
        CSRW(mepc, epc);
    } else { //中断处理
        cause = cause & 0x7fffffff;
        switch(cause){
            case 7: //machine timer interrupt
                timer_handler(f);
                break;
        }
    }
}
```





异常、中断和特权模式



- QEMU
- 在QEMU中运行程序
- 特权模式
- 异常
- 定时器中断